Bài tập Toán 02 — Tập hợp, Jaccard và đồ thị sinh viên

1. Đề bài tiếng Anh

1. Analyze Student Interests: list all unique subjects, identify students interested in both Python and AI, and identify students interested in Python or Math.
2. Compare Two Students: select two valid and different students; find their subject intersection, union, and subjects studied only by the first student.
3. Calculate Jaccard Similarity: calculate the size of the intersection divided by the size of the union, and handle an empty union.
4. Find the Most Similar Pair: evaluate every unordered pair of different students once and find the pair with the highest Jaccard score.
5. Recommend Study Partners: use Jaccard threshold 0.5, exclude the student themselves, and list suitable partners.
6. Build a Simple Student Graph: create an undirected graph linking students when their Jaccard similarity is at least 0.5.

2. Đề bài tiếng Việt

1. Phân tích sở thích học tập: liệt kê các môn học khác nhau, tìm người học cả Python và AI, và tìm người học Python hoặc Math.
2. So sánh hai sinh viên: chọn hai học viên hợp lệ và khác nhau; tìm giao, hợp và các môn chỉ người thứ nhất học.
3. Tính độ tương đồng Jaccard: lấy số môn chung chia cho số môn thuộc hợp; xử lý trường hợp hợp rỗng.
4. Tìm cặp giống nhau nhất: xét mỗi cặp hai học viên một lần, tìm cặp có Jaccard lớn nhất.
5. Gợi ý bạn học: lấy ngưỡng Jaccard 0,5; không gợi ý chính người đang xét; liệt kê các bạn phù hợp.
6. Tạo đồ thị sinh viên đơn giản: nối hai người bằng cạnh vô hướng khi Jaccard từ 0,5 trở lên.

3. Code Python có chú thích

```
"""Lab Math 02 — set operations, Jaccard similarity and student graphs.

Reconstructed and improved from PythoClass/2. Math/labs2.ipynb.
Run: python SOLUTION_ANNOTATED.py
All example names are synthetic; no user input is required.
"""
from itertools import combinations  # Generate unordered pairs without repetition.

# Map each synthetic student to a SET of subjects: duplicates are impossible.
# The subject choices reproduce the patterns in the owner's six-row demo.
STUDENTS = {
    "Student A": {"Python", "AI", "Math"},
    "Student B": {"Python", "AI", "SQL"},
    "Student C": {"Python", "SQL"},
    "Student D": {"Java", "Web"},
    "Student E": {"Python", "AI", "Math"},
    "Student F": {"AI", "Math", "Python"},
}


def analyse_interests(students):  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Task 1: compute unique subjects and the two requested filters."""
```

```
    # set().union(*iterable) joins every subject set into one unique collection.
    all_subjects = set().union(*students.values())
    # 'and' keeps people who study BOTH Python and AI.
    python_and_ai = [name for name, subjects in students.items()
                     if "Python" in subjects and "AI" in subjects]  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
    # 'or' keeps people who study AT LEAST ONE of Python and Math.
    python_or_math = [name for name, subjects in students.items()
                      if "Python" in subjects or "Math" in subjects]  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
    # Sorting gives reproducible output even though sets have no fixed order.
    return sorted(all_subjects), python_and_ai, python_or_math  # Trả kết quả về nơi gọi hàm.


def compare_students(students, first, second):  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Task 2: inspect intersection, union and directional difference."""
    # Reject unknown identifiers before attempting dictionary indexing.
    if first not in students or second not in students:  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
        raise ValueError("Both student names must exist.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    if first == second:  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
        raise ValueError("Choose two distinct students.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    left, right = students[first], students[second]
    return {
        "shared": sorted(left & right),  # Subjects common to BOTH students.
        "combined": sorted(left | right),  # Subjects studied by EITHER student.
        "only_first": sorted(left - right),  # Subjects absent from second.
    }


def jaccard_similarity(left, right):  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Task 3: fraction of unique shared subjects among unique subjects."""
    both = left & right  # Set intersection.
    either = left | right  # Set union.
    # Two empty sets have no shared subjects here; choose 0 rather than /0.
    return len(both) / len(either) if either else 0.0  # Trả kết quả về nơi gọi hàm.


def most_similar_pair(students):  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Task 4: compare all unordered pairs, preserving first winning tie."""
    best_pair = None  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
    best_score = -1.0  # Lower than every legal Jaccard value [0, 1].
    for first, second in combinations(students, 2):  # Duyệt qua các phần tử hoặc cặp phần tử.
        score = jaccard_similarity(students[first], students[second])  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
        if score > best_score:  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
            best_pair, best_score = (first, second), score
    return best_pair, best_score  # Trả kết quả về nơi gọi hàm.


def recommend_partners(students, student, threshold=0.5):  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Task 5: return everyone other than student with similarity >= limit."""
    if student not in students:  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
        raise ValueError("Unknown student.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    if not 0.0 <= threshold <= 1.0:  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
        raise ValueError("Threshold must be between 0 and 1.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    matches = []  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
    for other, subjects in students.items():  # Duyệt qua các phần tử hoặc cặp phần tử.
        if other == student:  # Never recommend a student to themselves.
            continue
        score = jaccard_similarity(students[student], subjects)  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
        if score >= threshold:  # Inclusive as in the original notebook.
```

```
            matches.append((other, score))
    # Rank by similarity, then alphabetically when scores are identical.
    return sorted(matches, key=lambda item: (-item[1], item[0]))  # Trả kết quả về nơi gọi hàm.


def build_graph(students, threshold=0.5):  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Task 6: symmetric adjacency list; isolated vertices remain visible."""
    if not 0.0 <= threshold <= 1.0:  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
        raise ValueError("Threshold must be between 0 and 1.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    graph = {name: [] for name in students}  # Every student becomes a node.
    for first, second in combinations(students, 2):  # Duyệt qua các phần tử hoặc cặp phần tử.
        score = jaccard_similarity(students[first], students[second])  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
        if score >= threshold:  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
            graph[first].append(second)  # Undirected edge in both directions.
            graph[second].append(first)
    return {name: sorted(neighbors) for name, neighbors in graph.items()}  # Trả kết quả về nơi gọi hàm.


def main():  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Show each result and assert easy-to-check reference expectations."""
    subjects, both, either = analyse_interests(STUDENTS)
    print("Task 1 - All subjects:", ", ".join(subjects))  # In kết quả để đối chiếu với đề bài.
    print("Python AND AI:", ", ".join(both))  # In kết quả để đối chiếu với đề bài.
    print("Python OR Math:", ", ".join(either))  # In kết quả để đối chiếu với đề bài.

    compared = compare_students(STUDENTS, "Student A", "Student B")
    print("Task 2 - Common:", compared["shared"])  # In kết quả để đối chiếu với đề bài.
    print("Union:", compared["combined"])  # In kết quả để đối chiếu với đề bài.
    print("Only Student A:", compared["only_first"])  # In kết quả để đối chiếu với đề bài.

    score = jaccard_similarity(STUDENTS["Student A"], STUDENTS["Student E"])  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
    print("Task 3 - Jaccard(A, E):", f"{score:.2f}")  # In kết quả để đối chiếu với đề bài.
    if not (score == 1.0): raise ValueError("Kiểm tra dữ liệu không đạt")  # These two students have identical interest sets.

    pair, highest = most_similar_pair(STUDENTS)
    print("Task 4 - Best pair:", pair, "score", f"{highest:.2f}")  # In kết quả để đối chiếu với đề bài.
    if not (highest == 1.0): raise ValueError("Kiểm tra dữ liệu không đạt")  # Duplicate interest sets give perfect similarity.

    matches = recommend_partners(STUDENTS, "Student A", threshold=0.5)  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
    print("Task 5 - Partners:", matches)  # In kết quả để đối chiếu với đề bài.
    if not (all(s >= 0.5 for _, s in matches)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra điều kiện bằng if thay cho assert.

    graph = build_graph(STUDENTS, threshold=0.5)  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
    print("Task 6 - Student graph:")  # In kết quả để đối chiếu với đề bài.
    for name, neighbors in graph.items():  # Duyệt qua các phần tử hoặc cặp phần tử.
        print(f"  {name}: {', '.join(neighbors) if neighbors else 'No connections'}")  # In kết quả để đối chiếu với đề bài.
    if not (graph["Student D"] == []): raise ValueError("Kiểm tra dữ liệu không đạt")  # Java/Web shares none of these subjects.


if __name__ == "__main__":  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
    main()  # Do not run the sample automatically when imported.
```

4. Giải thích nội dung và cách giải

Câu 1

Tập hợp (set) loại phần tử trùng; từ điển `STUDENTS` lưu mỗi người và tập các môn. `set().union(*students.values())` lấy tất cả môn xuất hiện. `and` yêu cầu hai môn cùng có mặt; `or` chỉ cần một trong hai.

Câu 2

Giao (`&`) lấy phần tử có ở cả hai tập, **hợp (`|`)** lấy phần tử có ít nhất một tập, **hiệu (`-`)** lấy các môn bên trái nhưng không ở bên phải. Trước khi tra cứu phải kiểm tra tên học viên hợp lệ, khác nhau.

Câu 3

`jaccard_similarity(left,right)` tính `len(left & right) / len(left | right)`. Ví dụ hai học viên có đúng ba môn giống nhau thì giao/hợp = 3/3 = 1.0 (100%). Với hợp rỗng, lời giải chọn 0 để không chia cho 0.

Câu 4

`combinations(students, 2)` sinh mỗi cặp khác nhau đúng một lần, không đếm lặp A–B và B–A. So Jaccard của từng cặp và giữ điểm cao nhất.

Câu 5

Ngưỡng `threshold=0.5` nghĩa là từ 50% tương đồng trở lên. Bỏ qua `other == student`, lấy các đối tượng đạt ngưỡng và sắp điểm giảm dần bằng `sorted(..., key=...)`.

Câu 6

Đồ thị lưu bằng **adjacency list** (danh sách kề): mỗi học viên là một nút, nếu hai học viên đủ giống nhau thì thêm tên mỗi người vào danh sách của người kia; vì vậy cạnh là **vô hướng**. Giữ cả các nút không có kết nối.

Đối chiếu nguồn và hướng dẫn chạy

Notebook gốc chứa các tiêu đề Task 1–6 và code mẫu. Lời giải đầy đủ là `labs/pythoclass/LAB-MATH-02-sets-jaccard/SOLUTION.py`, đã được cấu trúc lại từ notebook, sử dụng tên giả lập để không công khai dữ liệu cá nhân. Phần đề chi tiết bên dưới được diễn giải từ các task và code, không nhận là bản dịch nguyên văn một đề Word.

Khác biệt trình bày: Các điều kiện `assert` có trong `SOLUTION.py` được đổi thành `if not ...: raise ValueError(...)` để người mới hiểu cách kiểm tra rõ ràng hơn; không đổi mục tiêu tính toán.

Nguồn bài tập: `PythoClass/2. Math/labs2.ipynb` trên repo `nguyenletrian/PythoClass`.